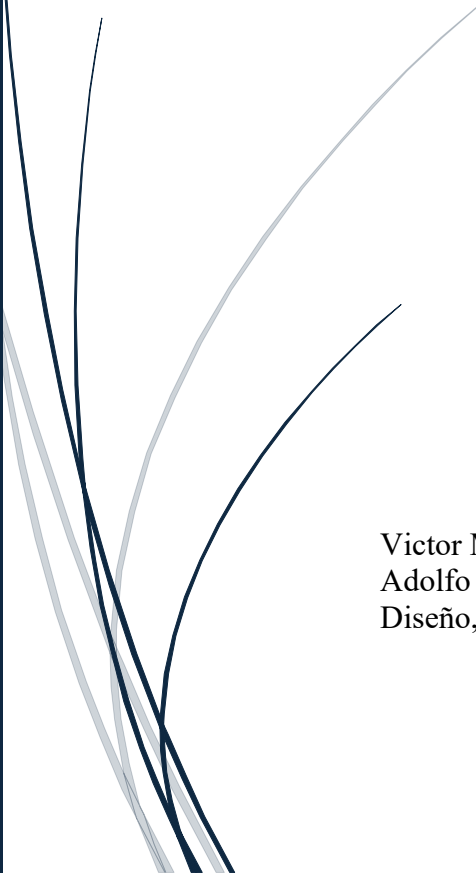




14-9-2026

Sequential Multiplier

Lab1



Victor Manuel Sandoval Maciel
Adolfo Meza Ibarra
Diseño, verificación y validación de sistemas digitales

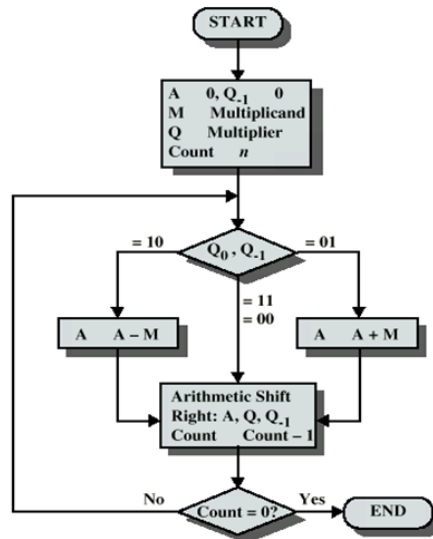
Content

Introducción.....	2
Diagrama.	3
Módulos.....	5
Mux2to1.	5
Datapath.....	5
Booth_result.	5
FSM_Mealy.....	6
Códigos.....	7
Sequential_Multiplier – Top.....	7
Mux2to1_mod – Mul_Or_Square	8
FSM_Mealy– Booth_controller.....	8
Booth_datapath – Booth_magic	10
Booth_result– Booth_result.....	11
Result_display– Display	12
Diagrama RTL	13
Implementación.	15
Pines utilizados.	15
Pin Planner.....	16
Pruebas en implementación.....	17
Automatizacion en tb de todas las pruebas.....	20
Código	20
Recursos.	23

Introducción.

A través del algoritmo de booth se plantea realizar e implementar un multiplicador secuencial con entradas de 5 bits signadas, definido por 5 switches para cada dato, en 3 botones se verán Start, Reset y Square. Dicho multiplicador tomara como señal de reloj la salida de un PLL a 5MHz, cuya entrada es la señal del reloj interno de 50 MHz.

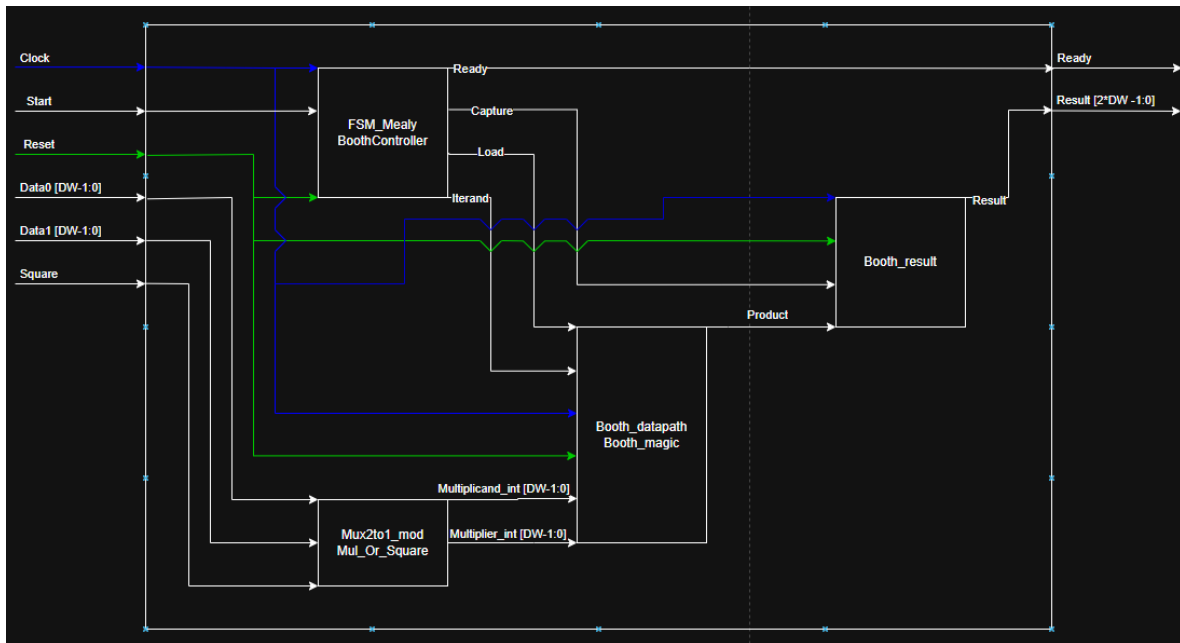
El algoritmo de booth tiene el siguiente flujo



El flujo es sencillo.

1. Se inician variables.
 - a. Multiplicando con valor X
 - b. Multiplicador con valor X
 - c. Acumulador en cero
 - d. Contador en cero
2. If contador < DW
 - a. Si. Se continua con el flujo
 - b. No. Se termina, y muestra el resultado
3. Se evalua Q_0 y Q_{-1}
 - a. Si $Q_0 = 1$ y $Q_{-1} = 0$
 - i. Acumulador = Acumulador - Mutiplicand
 - b. Si $Q_0 = 0$ y $Q_{-1} = 0$ o si $Q_0 = 1$ y $Q_{-1} = 1$
 - i. No se hace nada y Acumulador queda intacto
 - c. Si $Q_0 = 0$ y $Q_{-1} = 1$
 - i. Acumulador = Acumulador + Mutiplicand

Diagrama.



En el diagrama planeado mostramos los 4 módulos principales y los pensados para llevar a cabo el proyecto .

Los modulos principales, ignorando los necesario para mostrar el resultado en los 7 segmentos, son:

Selector

Decide si multiplicamos dos números distintos o uno por sí mismo (para lo de square). Una salida siempre entrega el primer operando y la otra entrega el segundo, salvo cuando está activo el modo square, donde repite el primero. Resolverlo aquí y no adentro del datapath nos ahorra hardware.

Datapath

El algoritmo de Booth y es el único bloque que hace aritmética. Tiene un registro de doce bits partido en tres: arriba va el acumulador con signo, en medio el multiplicador que se va recorriendo a la derecha, y el bit de hasta abajo guarda el valor auxiliar que Booth necesita para arrancar.

En cada iteración revisa los dos bits de abajo para decidir qué hacer. Si son cero y uno, se está acabando una cadena de unos y hay que sumar el multiplicando. Si son uno y cero, empieza una cadena y hay que restarlo. En los otros dos casos la cadena sigue igual y el acumulador no se toca. Después de decidir, todo el registro se recorre un lugar a la derecha.

Usamos un solo sumador para las dos operaciones: la resta la hacemos en complemento a dos, invirtiendo el multiplicando y metiendo la propia señal de control como acarreo de entrada. Cuando Booth decide no operar, un multiplexor tira la salida del sumador en vez de que agreguemos una segunda unidad aritmética.

El desplazamiento tiene que ser aritmético y no lógico. Declaramos la señal como signed para que al recorrerse repita el bit de signo en lugar de meter ceros. Con desplazamiento lógico fallan todos los casos negativos.

Al ancho del acumulador le pusimos un bit más del que uno esperaría porque al multiplicar el negativo más grande por sí mismo el resultado necesita nueve bits de magnitud. Con el ancho normal ese caso se desborda y entrega basura.

Result

Guarda el producto cuando la máquina de estados lo indica y lo mantiene quieto el resto del tiempo. Del registro del datapath se queda nada más con los bits de en medio, tirando el bit extra de arriba y el auxiliar de abajo, que es lo mismo que el desplazamiento final del pseudocódigo del enunciado.

Le conectamos la versión combinacional del producto y no la de registro. Esto es porque Capture se levanta durante la última iteración y no un ciclo después, así que en ese momento el valor bueno es el combinacional. Si le conectas el registrado, tres por cinco te da treinta en lugar de quince

Mientras Capture está en cero el registro conserva lo que tenía, con lo que se cumple que el resultado no cambie mientras la operación va en curso.

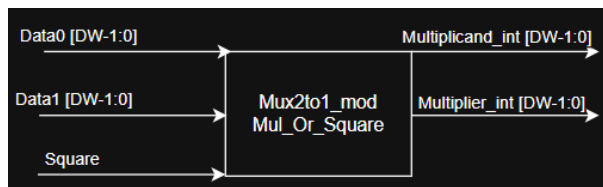
Controller

Controla la secuencia de la operación y genera las señales del datapath: Load para cargar los operandos, Iterate para cada paso del algoritmo, Capture para guardar el resultado y Ready para avisar que el sistema está libre.

Solo usa dos estados, reposo y ejecución, más un contador que lleva las iteraciones. Es Mealy porque las salidas dependen del estado y también de la entrada: estando en reposo, si llega Start, la señal de carga sube y Ready baja en ese mismo ciclo sin esperar al siguiente flanco.

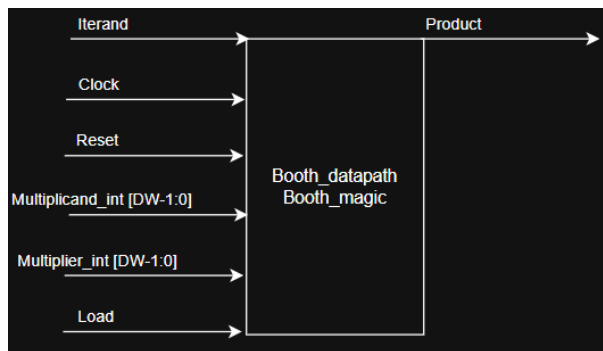
Módulos.

Mux2to1.



Este es básicamente un Mux2to1 que con ayuda de “Square” podemos seleccionar si Data0 se multiplica con Data1 o por si mismo.

Datapath.



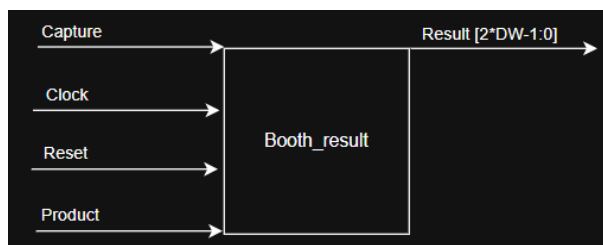
El “Datapath” se encarga de realizar las operaciones internas del algoritmo de Booth.

Relacionando con el flujo del funcionamiento de Booth algorithm es:

- Cargar el multiplicando y el multiplicador al iniciar la operación.
- Revisar los bits Q_0 y Q_{-1} del producto temporal.
- Según esos bits, decidir si el acumulador debe sumar, restar o mantenerse igual.
- Usar un sumador/restador para modificar el acumulador.
- Realizar el desplazamiento aritmético a la derecha en cada iteración.
- Por ultimo actualizar Product con el resultado temporal de cada paso.

En resumen, la FSM decide cuándo iterar y el Datapath realiza qué operación matemática corresponde en cada iteración.

Booth_result.



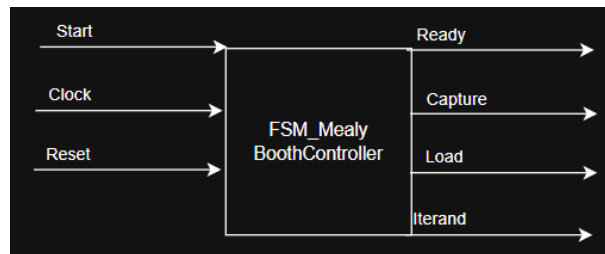
Booth_result se encarga de capturar y conservar el resultado final de la multiplicación.

Su función es.

- Recibe el valor calculado por el Datapath.
- Espera a que la señal Capture se active.
- Cuando Capture = 1, toma los bits útiles del producto temporal y los guarda en Result. Mientras Capture = 0, mantiene el resultado anterior sin cambios.
- También limpia Result cuando se activa el reset.

En resumen, el Datapath calcula el producto y Booth_result lo registra y lo presenta como salida final del sistema.

FSM_Mealy.



La máquina de estados controla el orden en que se ejecuta la multiplicación.

Está formada por dos estados:

- IDLE: el sistema espera una nueva operación. Ready permanece activo. Cuando Start = 1, se cargan los operandos mediante Load y comienza el proceso.
- RUN: se ejecutan las iteraciones del algoritmo de Booth. En cada ciclo se activa Iterate y se incrementa el contador. Cuando se alcanza la última iteración, se activa Capture para guardar el resultado y la máquina regresa a IDLE.

En resumen, la máquina de estados organiza el flujo esperar, cargar, iterar, capturar, volver a esperar.

Códigos.

Sequential_Multiplier – Top

```
module Sequential_multiplier#( parameter int DW = 5
)(
    input logic Start,
    input logic Reset,
    input logic Clock,
    input logic Square,
    input logic [DW-1:0] Data0,
    input logic [DW-1:0] Data1,

    output logic Ready,
    output logic [(2*DW)-1:0] Result,

    output logic [6:0] HEX0,
    output logic [6:0] HEX1,
    output logic [6:0] HEX2,
    output logic [6:0] HEX3
);

    logic [DW-1:0] Multiplicand_w;
    logic [DW-1:0] Multiplier_w;

    logic [(2*DW)+1:0] Product_w;
    logic [(2*DW)+1:0] Product_next_w;

    logic Iterate_w;
    logic Load_w;
    logic Capture_w;

    logic Clock_5MHz_w;
    logic PLL_locked_w;
    logic Reset_system_w;

    assign Reset_system_w = Reset & PLL_locked_w;

    pll Clock_generator (
        .refclk    (Clock),
        .rst       (~Reset),
        .outclk_0  (Clock_5MHz_w),
        .locked    (PLL_locked_w)
    );

    Mux2to1_mod #( .DW(DW)
    ) Mul_Or_Square (
        .Input0(Data0),
        .Input1(Data1),
        .Sel(~Square),
        .Output_fixed(Multiplicand_w),
        .Output2(Multiplier_w)
    );

    FSM_Mealy #( .DW(DW)
    ) Booth_controller (
        .Clock(Clock_5MHz_w),
        .Reset(Reset_system_w),
        .Start(~Start),

        .Ready(Ready),
        .Iterate(Iterate_w),
        .Load(Load_w),
        .Capture(Capture_w)
    );

    Booth_datapath #( .DW(DW)
    ) Booth_magic (
        .Reset(Reset_system_w),
        .Clock(Clock_5MHz_w),
        .Multiplicand_init(Multiplicand_w),
        .Multiplier_init(Multiplier_w),
        .Iterate(Iterate_w),
```

```

        .Load(Load_w),

        .Product(Product_w),
        .Product_next(Product_next_w)
    );

    Booth_result #( .DW(DW)
    ) Booth_result (
        .Clock(Clock_5MHz_w),
        .Reset(Reset_system_w),
        .Product(Product_next_w),
        .Capture(Capture_w),

        .Result(Result)
    );

    Result_display #( .DW(DW)
    ) Display (
        .Result(Result),

        .HEX0(HEX0),
        .HEX1(HEX1),
        .HEX2(HEX2),
        .HEX3(HEX3)
    );

endmodule

```

Mux2to1_mod – Mul_Or_Square

```

module Mux2to1_mod #( parameter int DW
)
(
    input logic [DW-1:0] Input0,
    input logic [DW-1:0] Input1,
    input logic Sel,

    output logic [DW-1:0] Output_fixed,
    output logic [DW-1:0] Output2
);
// Salida Data0 fija
assign Output_fixed = Input0;

// Square define la segunda salida, en caso de ser
// Sel==1: DATA0*DATA0
// Sel==0: DATA0*DATA1
always_comb begin
    if (Sel == 1'b1) begin
        Output2 = Input0;
    end
    else begin
        Output2 = Input1;
    end
end
endmodule

```

FSM_Mealy– Booth_controller

```

module FSM_Mealy #( parameter int DW
)
(
    input logic Clock,
    input logic Reset,
    input logic Start,

    output logic Ready,
    output logic Iterate,
    output logic Load,
    output logic Capture
);
// Cantidad de Iteraciones
localparam logic [2:0] Last_iteration = DW - 1;

```

```

// Estados definidos
// Ideal. esperando un nuevo start
// Load = 0;
// Iterate =0;
// Capture =0;
// Ready = 1;
// Load. Cargar e inicializar operandos
// Load = 1;
// Iterate =0;
// Capture =0;
// Ready = 0;
// Run. Ejecutando Booth algorithm
// Load = 0;
// Iterate =1;
// Capture =0;
// Ready = 0;
// Done. Se terminaron las iteraciones y se tiene el resultado listo para mostrar
// Load = 0;
// Iterate =0;
// Capture =1;
// Ready = 0;

// Estados
typedef enum logic {
    IDLE,
    RUN
} state_t;

// Desplazamientos entre estados e contador para iteraciones
state_t Current_state;
state_t Next_state;

logic [2:0] Iteration_count;

// Bloque sincrono de decision por iteracion segun Booth
always_ff @(posedge Clock or negedge Reset) begin
    if (Reset == 1'b0) begin
        Current_state <= IDLE;
        Iteration_count <= 1'b0;
    end
    else begin
        Current_state <= Next_state;
        if (Current_state == IDLE) begin
            Iteration_count <= '0;
        end
        else if (Current_state == RUN) begin
            if (Iteration_count == Last_iteration)
                Iteration_count <= '0;
            else
                Iteration_count <= Iteration_count + 1'b1;
            end
        end
    end
end

// Bloque asincrono para el cambio de estados. Desplazamientos entre ellos
always_comb begin
    Next_state = Current_state;
    case (Current_state)
        IDLE: begin
            if (Start == 1'b1)
                Next_state = RUN;
            end
        RUN: begin
            if (Iteration_count == Last_iteration)
                Next_state = IDLE;
            else
                Next_state = RUN;
            end
        default: begin
            Next_state = IDLE;
        end
    endcase
end

// Efectos del estado actual
always_comb begin
    Load = 1'b0;

```

```

        Iterate = 1'b0;
        Capture = 1'b0;
        Ready   = 1'b0;

        case (Current_state)
            IDLE: begin
                Ready = 1'b1;
                if (Start == 1'b1) begin
                    Load = 1'b1;
                    Ready = 1'b0;
                end
            end
            RUN: begin
                Iterate = 1'b1;
                if (Iteration_count == Last_iteration)
                    Capture = 1'b1;
                else begin
                    Capture = 1'b0;
                end
            end
            default: begin
                Ready = 1'b1;
            end
        endcase
    end
endmodule

```

Booth_datapath – Booth_magic

```

module Booth_datapath #( parameter int DW = 5
)
(
    input logic      Clock,
    input logic      Reset,
    input logic      Load,
    input logic      Iterate,
    input logic [DW-1:0] Multiplicand_init,
    input logic [DW-1:0] Multiplier_init,

    output logic [2*DW+1:0] Product,
    output logic [2*DW+1:0] Product_next
);

// Distribucion de bits de Product (2*DW+2 bits en total):
//   [2*DW+1 : DW+1] acumulador A, con signo. Lleva DW+1 bits y no DW porque
//                   -2^(DW-1) al cuadrado se desbordaria con DW bits.
//   [DW:1]          multiplicador Q, que se recorre a la derecha en cada paso
//   [0]             Q(-1), el bit auxiliar que Booth necesita para arrancar
// El producto final son los 2*DW bits centrales: Product[2*DW : 1]

    logic signed [DW:0]      Multiplicand_r;

    logic                  Operate_w;
    logic                  Sub_w;

// Nodos del unico sumador/restador que tiene el multiplicador
    logic signed [DW:0]      Acc_w;
    logic signed [DW:0]      Addend_w;
    logic signed [DW:0]      Addsub_w;
    logic signed [DW:0]      Acc_op_w;

    logic signed [2*DW+1:0]  Product_pre_shift_w;
    logic                  [2*DW+1:0] Product_d_w;

    assign Acc_w = Product[2*DW+1 : DW+1];

// Booth decide en funcion de los dos bits inferiores, Q(0) y Q(-1):
//   01 -> termina una cadena de unos -> sumar el multiplicando
//   10 -> empieza una cadena de unos -> restar el multiplicando
//   00 y 11 -> la cadena continua, el acumulador se queda igual
    always_comb begin
        case (Product[1:0])
            2'b01: begin Operate_w = 1'b1; Sub_w = 1'b0; end
            2'b10: begin Operate_w = 1'b1; Sub_w = 1'b1; end
        endcase
    end

```

```

        default: begin Operate_w = 1'b0; Sub_w = 1'b0; end
    endcase
end

// Un solo sumador resuelve las dos operaciones. La resta se hace en complemento a
// 2: se invierte el multiplicando y el propio Sub_w entra como acarreo inicial,
// de manera que A - M se calcula como A + (~M) + 1.
assign Addend_w = Sub_w ? ~Multiplicand_r : Multiplicand_r;
assign Addsub_w = Acc_w + Addend_w + Sub_w;

// Cuando Booth no opera se descarta la salida del sumador con este mux, en vez de
// agregar una segunda unidad aritmetica
assign Acc_op_w = Operate_w ? Addsub_w : Acc_w;

// Al recorrer a la derecha se repite el bit de signo en lugar de introducir un
// cero. Con un desplazamiento logico fallarian todos los casos negativos, y solo
// resulta aritmetico porque Product_pre_shift_w esta declarado signed.
assign Product_pre_shift_w = { Acc_op_w, Product[DW:0] };
assign Product_next
    = Product_pre_shift_w >>> 1;

always_comb begin
    if (Load) begin
        Product_d_w = { {(DW+1){1'b0}}, Multiplier_init, 1'b0 };
    end
    else if (Iterate) begin
        Product_d_w = Product_next;
    end
    else begin
        Product_d_w = Product;
    end
end

// Reset asincrono y activo bajo
always_ff @(posedge Clock or negedge Reset) begin
    if (Reset == 1'b0) begin
        Multiplicand_r <= '0;
        Product <= '0;
    end
    else begin
        // $signed obliga a que la extension a DW+1 bits repita el bit de signo
        if (Load) begin
            Multiplicand_r <= $signed(Multiplicand_init);
        end
        Product <= Product_d_w;
    end
end
endmodule

```

Booth_result– Booth_result

```

module Booth_result #( parameter int DW
)
(
    input logic Clock,
    input logic Reset,
    input logic Capture,
    input logic [(2*DW)+1:0] Product,

    output logic [(2*DW)-1:0] Result
);

// El registro Product del datapath lleva un bit de mas por arriba y el bit Q(-1)
// de Booth por abajo. El producto util son los 2*DW bits centrales, que equivalen
// al "result = product_temporal >> 1" del pseudocodigo de la especificacion.
//
// Con Capture en cero se conserva el valor anterior, porque la especificacion pide
// que Result permanezca estable mientras la operacion sigue en curso.

always_ff @(posedge Clock or negedge Reset) begin
    if (Reset == 1'b0) begin
        Result <= '0;
    end
    else if (Capture) begin
        Result <= Product[2*DW : 1];
    end
end

```

```

        end
    end
endmodule

```

Result_display– Display

```

module Result_display #( parameter int DW = 5
)
(
    input logic [(2*DW)-1:0] Result,

    output logic [6:0] HEX0,
    output logic [6:0] HEX1,
    output logic [6:0] HEX2,
    output logic [6:0] HEX3
);

// Signo y magnitud
    logic Sign_w;
    logic [(2*DW)-1:0] Magnitude_full_w;
    logic [8:0] Magnitude_w;
    logic [7:0] bin;

// Digitos BCD
    logic [3:0] bcd_centenas;
    logic [3:0] bcd_decenas;
    logic [3:0] bcd_unidades;

// Senales internas de Double Dabble
    logic [3:0] c1, c2, c3, c4, c5, c6, c7;

// Signo y valor absoluto
    assign Sign_w = Result[(2*DW)-1];
    assign Magnitude_w = Magnitude_full_w[8:0];
    assign bin = Magnitude_w[7:0];

    always_comb begin
        if (Sign_w == 1'b1) begin
            Magnitude_full_w = ~Result + 1'b1;
        end
        else begin
            Magnitude_full_w = Result;
        end
    end

// Double Dabble
    BCD u1 ( .in ({1'b0, bin[7:5]}), .out(c1) );
    BCD u2 ( .in ({c1[2:0], bin[4]}), .out(c2) );
    BCD u3 ( .in ({c2[2:0], bin[3]}), .out(c3) );
    BCD u4 ( .in ({c3[2:0], bin[2]}), .out(c4) );
    BCD u5 ( .in ({1'b0, c1[3], c2[3], c3[3]}), .out(c5) );
    BCD u6 ( .in ({c4[2:0], bin[1]}), .out(c6) );

    BCD u7 ( .in ({c5[2:0], c4[3]}), .out(c7) );

// BCD final

    always_comb begin
// Para DW = 5, el unico valor cuya magnitud
// necesita 9 bits es 256.
        if (Magnitude_w[8]==1) begin
            bcd_centenas = 4'd2;
            bcd_decenas = 4'd5;
            bcd_unidades = 4'd6;
        end
        else begin
            bcd_unidades = {c6[2:0], bin[0]};
            bcd_decenas = {c7[2:0], c6[3]};
            bcd_centenas = {2'b00, c5[3], c7[3]};
        end
    end
end

```

```

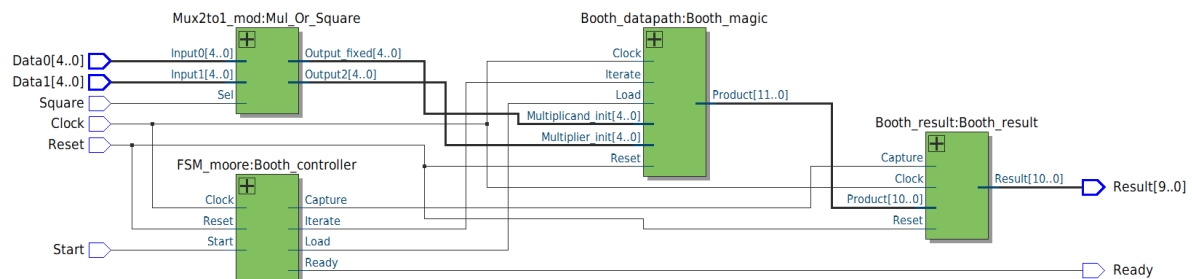
// BCD a 7 seg.
BinToSegs seg_units ( .in (bcd_unidades), .seg(HEX0) );
BinToSegs seg_tens ( .in (bcd_decenas), .seg(HEX1) );
BinToSegs seg_hundreds ( .in (bcd_cenetas), .seg(HEX2) );

// Signo
always_comb begin
    if (Sign_w)
        HEX3 = 7'b0111111; // "-"
    else
        HEX3 = 7'b1111111; // apagado
    end
end
endmodule

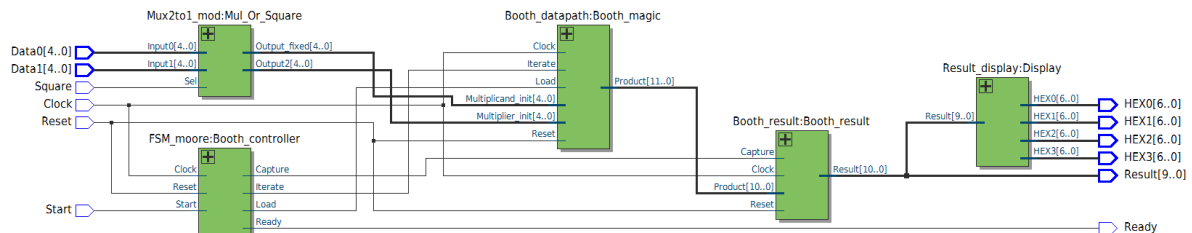
```

Diagrama RTL

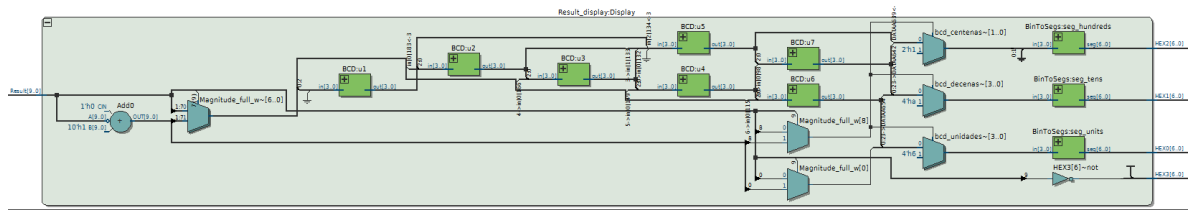
Antes de agregar BCD



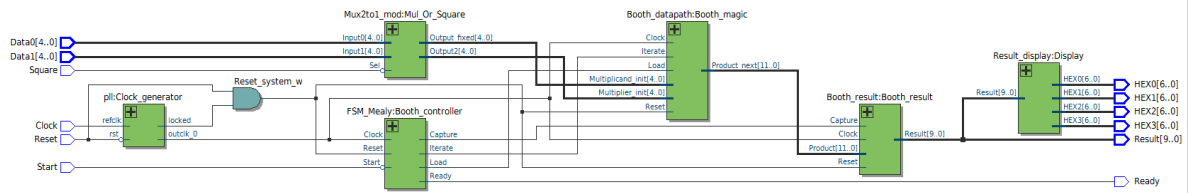
Después de agregar BCD



El interior de Result_Display muestra, como se realizo en la tarea 3 con algunos pequeños cambios, la instancia de los módulos previamente creados de BCD y bcd_”units” y uno nuevo para el manejo del signo y complemento a 2.



Después de agregar Pll.



Implementación.

Pines utilizados.

Parte	Pin	Signal
Switches		
SW[0]	PIN_AB30	Data0_0
SW[1]	PIN_Y27	Data0_1
SW[2]	PIN_AB28	Data0_2
SW[3]	PIN_AC30	Data0_3
SW[4]	PIN_W25	Data0_4
SW[5]	PIN_V25	Data1_0
SW[6]	PIN_AC28	Data1_1
SW[7]	PIN_AD30	Data1_2
SW[8]	PIN_AC29	Data1_3
SW[9]	PIN_AA30	Data1_4
Buttons		
KEY[0]	PIN_AJ4	Start
KEY[1]	PIN_AK4	Reset
KEY[2]	PIN_AA14	Square
KEY[3]	PIN_AA15	
Display		
HEX0	PIN_W17	Result0_a
HEX0	PIN_V18	Result0_b
HEX0	PIN_AG17	Result0_c
HEX0	PIN_AG16	Result0_d
HEX0	PIN_AH17	Result0_e
HEX0	PIN_AG18	Result0_f
HEX0	PIN_AH18	Result0_g
HEX1	PIN_AF16	Result1_a
HEX1	PIN_V16	Result1_b
HEX1	PIN_AE16	Result1_c
HEX1	PIN_AD17	Result1_d
HEX1	PIN_AE18	Result1_e
HEX1	PIN_AE17	Result1_f
HEX1	PIN_V17	Result1_g
HEX2	PIN_AA21	Result2_a
HEX2	PIN_AB17	Result2_b
HEX2	PIN_AA18	Result2_c
HEX2	PIN_Y17	Result2_d
HEX2	PIN_Y18	Result2_e
HEX2	PIN_AF18	Result2_f
HEX2	PIN_W16	Result2_g
HEX3	PIN_Y19	
HEX3	PIN_W19	
HEX3	PIN_AD19	
HEX3	PIN_AA20	
HEX3	PIN_AC20	
HEX3	PIN_AA19	
HEX3	PIN_AD20	Symbol
LED		
LED0	PIN_AA24	Ready
CLK		
CLOCK_50	PIN_AF14	Clock

Pin Planner.

IN	Start	Input	PIN A4	3B	B3B NO	PIN A4	2.5 V		12mA (default)			
IN	Square	Input	PIN AA14	3B	B3B NO	PIN AA14	2.5 V		12mA (default)			
OUT	Result[0]	Output				PIN AG22	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[1]	Output				PIN AE19	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[2]	Output				PIN AK19	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[3]	Output				PIN AG20	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[4]	Output				PIN AH24	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[5]	Output				PIN AC18	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[6]	Output				PIN AJ20	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[7]	Output				PIN AF19	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[8]	Output				PIN AG23	2.5 V (default)		12mA (default)	1 (default)		
OUT	Result[9]	Output				PIN AK28	2.5 V (default)		12mA (default)	1 (default)		
IN	Reset	Input	PIN AK4	3B	B3B NO	PIN AK4	2.5 V		12mA (default)			
OUT	Ready	Output	PIN AA24	5A	B5A NO	PIN AA24	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[0]	Output	PIN Y19	4A	B4A NO	PIN Y19	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[1]	Output	PIN W19	4A	B4A NO	PIN W19	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[2]	Output	PIN AD19	4A	B4A NO	PIN AD19	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[3]	Output	PIN AA20	4A	B4A NO	PIN AA20	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[4]	Output	PIN AC20	4A	B4A NO	PIN AC20	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[5]	Output	PIN AA19	4A	B4A NO	PIN AA19	2.5 V		12mA (default)	1 (default)		
OUT	HEX3[6]	Output	PIN AD20	4A	B4A NO	PIN AD20	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[0]	Output	PIN AA21	4A	B4A NO	PIN AA21	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[1]	Output	PIN AB17	4A	B4A NO	PIN AB17	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[2]	Output	PIN AA18	4A	B4A NO	PIN AA18	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[3]	Output	PIN Y17	4A	B4A NO	PIN Y17	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[4]	Output	PIN Y18	4A	B4A NO	PIN Y18	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[5]	Output	PIN AF18	4A	B4A NO	PIN AF18	2.5 V		12mA (default)	1 (default)		
OUT	HEX2[6]	Output	PIN W16	4A	B4A NO	PIN W16	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[0]	Output	PIN AF16	4A	B4A NO	PIN AF16	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[1]	Output	PIN V16	4A	B4A NO	PIN V16	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[2]	Output	PIN AE16	4A	B4A NO	PIN AE16	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[3]	Output	PIN AD17	4A	B4A NO	PIN AD17	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[4]	Output	PIN AE18	4A	B4A NO	PIN AE18	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[5]	Output	PIN AE17	4A	B4A NO	PIN AE17	2.5 V		12mA (default)	1 (default)		
OUT	HEX1[6]	Output	PIN V17	4A	B4A NO	PIN V17	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[0]	Output	PIN W17	4A	B4A NO	PIN W17	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[1]	Output	PIN V18	4A	B4A NO	PIN V18	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[2]	Output	PIN AG17	4A	B4A NO	PIN AG17	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[3]	Output	PIN AG16	4A	B4A NO	PIN AG16	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[4]	Output	PIN AH17	4A	B4A NO	PIN AH17	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[5]	Output	PIN AG18	4A	B4A NO	PIN AG18	2.5 V		12mA (default)	1 (default)		
OUT	HEX0[6]	Output	PIN AH18	4A	B4A NO	PIN AH18	2.5 V		12mA (default)	1 (default)		
IN	Data1[0]	Input	PIN V25	5B	B5B NO	PIN V25	2.5 V		12mA (default)			
IN	Data1[1]	Input	PIN AC28	5B	B5B NO	PIN AC28	2.5 V		12mA (default)			
IN	Data1[2]	Input	PIN AD30	5B	B5B NO	PIN AD30	2.5 V		12mA (default)			
IN	Data1[3]	Input	PIN AC29	5B	B5B NO	PIN AC29	2.5 V		12mA (default)			
IN	Data1[4]	Input	PIN AA30	5B	B5B NO	PIN AA30	2.5 V		12mA (default)			
IN	Data0[0]	Input	PIN AB30	5B	B5B NO	PIN AB30	2.5 V		12mA (default)			
IN	Data0[1]	Input	PIN Y27	5B	B5B NO	PIN Y27	2.5 V		12mA (default)			
IN	Data0[2]	Input	PIN AB28	5B	B5B NO	PIN AB28	2.5 V		12mA (default)			
IN	Data0[3]	Input	PIN AC30	5B	B5B NO	PIN AC30	2.5 V		12mA (default)			
IN	Data0[4]	Input	PIN W25	5B	B5B NO	PIN W25	2.5 V		12mA (default)			
IN	Clock	Input	PIN AF14	3B	B3B NO	PIN AF14	2.5 V		12mA (default)			

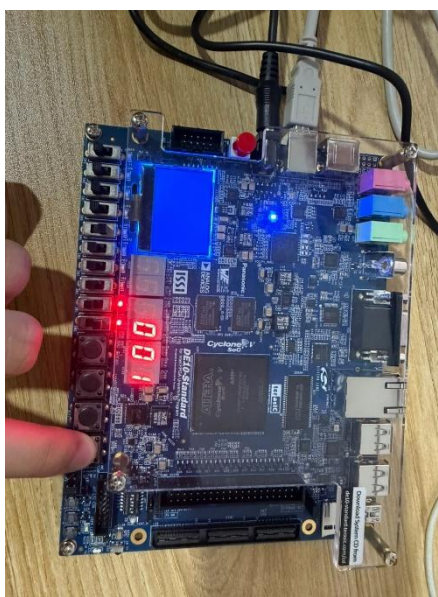
Pruebas en implementación



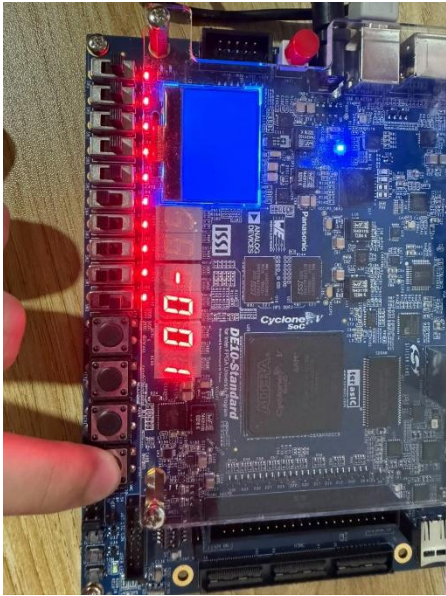
Operación 0×0



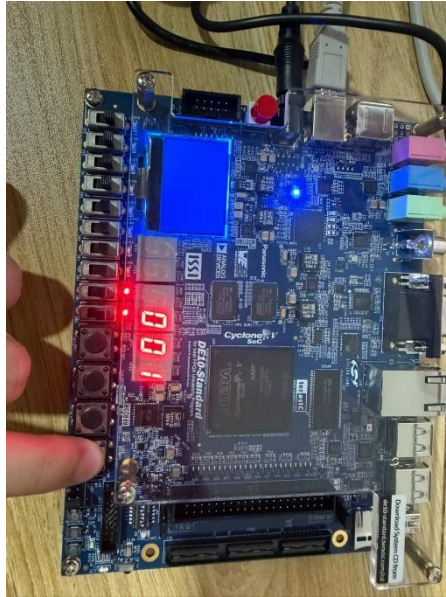
1 al cuadrado



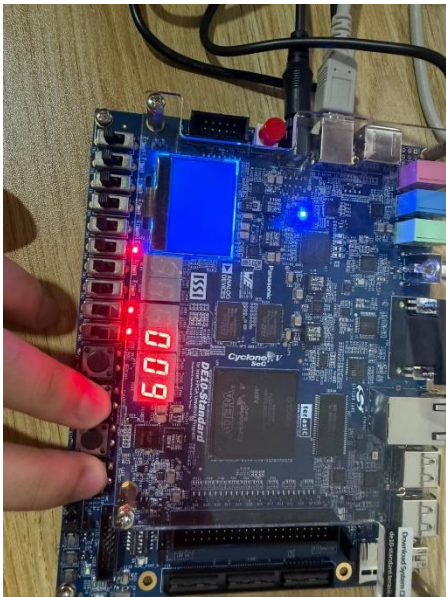
1x1



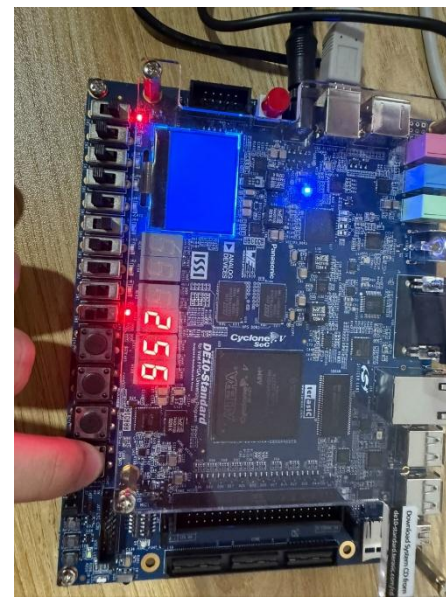
1x-1



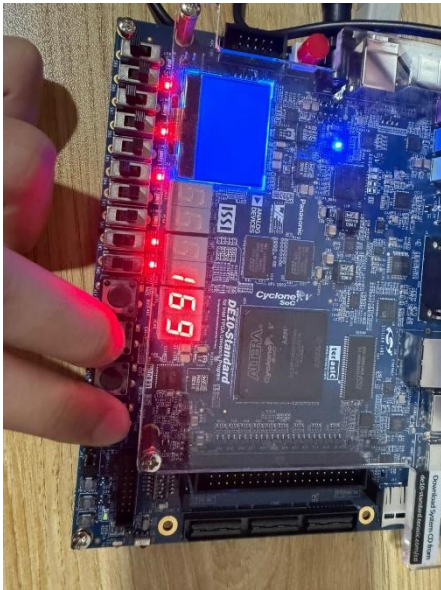
-1x-1



3 al cuadrado



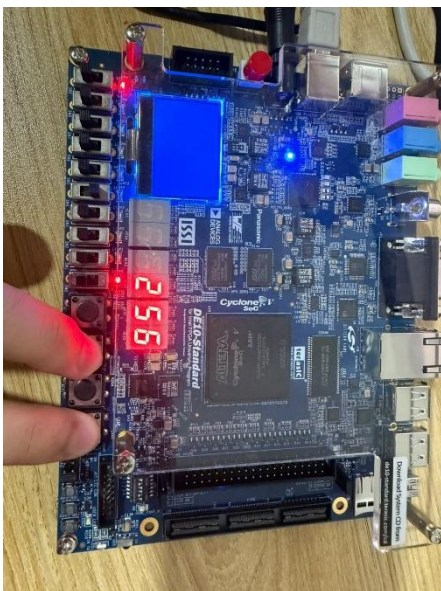
-16x-16



13 al cuadrado



13x11



-16 al cuadrado



15 al cuadrado

Automatizacion en tb de todas las pruebas

Para esto se realizo un ciclo en un archivo .sv en donde se verifica que el resultado de todas las operaciones dé el resultado correcto.

Se recorren todas las combinaciones entre los operandos desde -16 a 15, dando un total de 1024 multiplicaciones. Para esto en cada caso se sigue lo siguiente:

1. Carga un valor a Data0 y Data1
2. Se espera al ready que indique que podemos comenzar con la siguiente multiplicación
3. Se genera a la entrada Start un pulso
4. Una vez se tiene el resultado se vuelve a activar ready
5. Fuera de la implementación se calcula el valor de ambos datos y se almacena en expected
6. Se comparan ambos resultados el esperado y el obtenido en Result después de aplicarle un cast
7. Si son iguales se suma en uno al contado de pass, si fallo, se suma uno al contador de failed y se imprime la operación fallida.
8. Finalmente se imprime el total de operaciones, total de Passed y Failed como se ve a continuación.

Registro de pruebas.

```
VSIM 2> run -all
#
# =====
# MULTIPLICATION TEST FINISHED
# =====
# Total : 1024
# Passed :D : 1024
# Failed D: : 0
# =====
# RESULT: ALL TESTS PASSED
# ** Note: $stop : C:/Users/vitor/Desktop/Verificacion/Lab1/tb/Sequential_multiplier_tb.sv(134)
# Time: 1229101 ns Iteration: 0 Instance: /Sequential_multiplier_tb
# Break in Module Sequential_multiplier_tb at C:/Users/vitor/Desktop/Verificacion/Lab1/tb/Sequential_multiplier_tb.sv line 134
```

Código

```
`timescale 1ns / 1ps

module Sequential_multiplier_Auto_tb;

    parameter int DW = 5;

    logic Start;
    logic Reset;
    logic Clock;
    logic Square;

    logic [DW-1:0] Data0;
    logic [DW-1:0] Data1;

    logic Ready;
    logic [(2*DW)-1:0] Result;

    logic [6:0] HEX0, HEX1, HEX2, HEX3;

    int passed;
    int failed;
```

```

int a;
int b;

int expected;
int received;

Sequential_multiplier #(
    .DW(DW)
) dut (
    .Start(~Start),
    .Reset(~Reset),
    .Clock(Clock),
    .Square(~Square),

    .Data0(Data0),
    .Data1(Data1),

    .Ready(Ready),
    .Result(Result),

    .HEX0(HEX0),
    .HEX1(HEX1),
    .HEX2(HEX2),
    .HEX3(HEX3)
);

// Clock de 5 MHz
always #100 Clock = ~Clock;

initial begin

    // Inicialización
    Clock = 1'b0;
    Reset = 1'b0;
    Start = 1'b0;
    Square = 1'b0;

    Data0 = '0;
    Data1 = '0;

    passed = 0;
    failed = 0;

    // Reset
    #200;
    Reset = 1'b1;

    #200;

    // Pruebas exhaustivas
    for (a = -16; a <= 15; a = a + 1) begin
        for (b = -16; b <= 15; b = b + 1) begin

            Data0 = a;
            Data1 = b;
            Square = 1'b0;

            // Esperar a que el multiplicador esté disponible
            wait (Ready == 1'b1);

            // Lanzar Start
            @(negedge Clock);
            Start = 1'b1;

            @(negedge Clock);
            Start = 1'b0;

            // Esperar que procese
            wait (Ready == 1'b0);

            // Esperar que termine
            wait (Ready == 1'b1);

```

```

        // Leer después de que el resultado ya quedó registrado
        #1;

        expected = a * b;
        received = $signed(Result);

        if (received == expected) begin
            passed = passed + 1;
        end
        else begin
            failed = failed + 1;

            $display( "FAIL: %0d * %0d | Expected=%0d | Received=%0d", a, b, expected, received );
        end
    end
end

// Resumen
$display("");
$display("=====");
$display(" MULTIPLICATION TEST FINISHED");
$display("=====");
$display(" Total   : %0d", passed + failed);
$display(" Passed :D : %0d", passed);
$display(" Failed D: : %0d", failed);
$display("=====");

if (failed == 0)
    $display(" RESULT: ALL TESTS PASSED");
else
    $display(" RESULT: TEST FAILED");

$stop;

end

endmodule

```

Recursos.

[Booth Multiplier Explained \(With Examples\) | DigiSim.io](#)

[Booth's Multiplication Algorithm - GeeksforGeeks](#)